

```
"""
```

Parser de estrutura hierárquica de NRs (Normas Regulamentadoras).

Constrói uma árvore (capítulo > item > subitem > ... > anexo) a partir do PDF oficial, seguindo o schema validado em nr13_schema_exemplo.json.

Princípios de honestidade dos dados (resposta ao problema encontrado no JSON de exemplo anterior):

- Todo campo `content` vem literalmente do texto extraído, sem paráfrase.
- Toda referência cruzada em `references` só é incluída se casar com um padrão regex explícito, e o campo `source` descreve exatamente qual padrão bateu – nunca "explícita no texto" por inferência lógica.
- Nenhuma relação é inventada para ilustrar o schema.

```
"""
```

```
import pdfplumber
import re
import json
import sys
from dataclasses import dataclass, field, asdict
```

```
FOOTER_NOISE = {"Este texto não substitui o publicado no DOU"}
```

```
# case-sensitive de propósito: cabeçalho real é "ANEXO I DA NR-13" (tudo maiúscul
# Menções soltas no corpo do texto usam "Anexo II desta NR." (minúsculo em "nexo"
# e não devem ser confundidas com início de seção.
```

```
ROMAN_ANEXO_RE = re.compile(r"^ANEXO\s+([IVXLCDM]+) (?:\s+DA\s+NR-?\s*(\d+))?$")
```

```
GLOSSARIO_RE = re.compile(r"^GLOSS[ÁA]RIO\s*$", re.IGNORECASE)
```

```
LETTERED_ITEM_RE = re.compile(r"^[a-z]\)\s")
```

```
# referências: só o que bate padrão explícito
```

```
REF_NR_RE = re.compile(r"\bNR[\s-]?(\d{1,2})\b")
```

```
REF_SUBITEM_RE = re.compile(r"subitem\s+(\d{1,2})(?:\.\d{1,4}){1,6})", re.IGNORECA
```

```
REF_ANEXO_RE = re.compile(r"Anexo\s+([IVXLCDM]+)\s+desta\s+NR", re.IGNORECASE)
```

```
@dataclass
```

```
class Node:
```

```
    node_id: str
```

```
    document_id: str
```

```
    document_type: str
```

```
    chapter_number: str
```

```
    depth: int
```

```

chapter_path: str
title: str | None
content: str
has_table: bool = False
references: list = field(default_factory=list)

def extract_lines(pdf_path: str) -> list[str]:
    with pdfplumber.open(pdf_path) as pdf:
        pages_text = [p.extract_text() or "" for p in pdf.pages]
    raw = "\n".join(pages_text)
    lines = [l.strip() for l in raw.split("\n")]
    return [l for l in lines if l and l not in FOOTER_NOISE]

def make_heading_re(chapter_prefix: str) -> re.Pattern:
    # só reconhece números que começam pelo capítulo desta NR (ex.: "13")
    # evita falso positivo tipo "60 kPa" no início de linha
    return re.compile(rf"^{(re.escape(chapter_prefix))}(?:\.\d{{1,4}}){{0,6}})\s+(

def is_section_header(text: str) -> bool:
    """Heurística: título curto sem pontuação de fechamento de frase."""
    words = text.split()
    if len(words) > 8:
        return False
    if text.rstrip().endswith((".", ";", ":")):
        return False
    return True

def extract_references(content: str, self_doc: str, node_lookup: dict) -> list[di
    refs = []
    seen = set()

    for m in REF_NR_RE.finditer(content):
        num = m.group(1)
        target = f"NR-{int(num):02d}"
        if target == self_doc:
            continue # não conta autorreferência ao próprio documento como cross
        key = ("doc", target)
        if key in seen:
            continue
        seen.add(key)
        refs.append({
            "target_document": target,
            "target_node": None, # documento externo, não temos a árvore dele ca

```

```

        "relation": "referencia",
        "source": f"regex: menção direta a '{m.group(0)}' no texto"
    })

for m in REF_SUBITEM_RE.finditer(content):
    num = m.group(1)
    target_node_id = f"{self_doc.replace('-', ' ')}_{num}"
    key = ("subitem", num)
    if key in seen:
        continue
    seen.add(key)
    refs.append({
        "target_document": self_doc,
        "target_node": target_node_id if target_node_id in node_lookup else N
        "relation": "referencia",
        "source": f"regex: menção direta a 'subitem {num}' no texto"
    })

for m in REF_ANEXO_RE.finditer(content):
    roman = m.group(1)
    key = ("anexo", roman)
    if key in seen:
        continue
    seen.add(key)
    refs.append({
        "target_document": self_doc,
        "target_node": f"{self_doc.replace('-', ' ')}_ANEXO_{roman}",
        "relation": "referencia",
        "source": f"regex: menção direta a 'Anexo {roman} desta NR' no texto"
    })

return refs

def parse_nr(pdf_path: str, document_id: str, chapter_prefix: str) -> list[Node]:
    lines = extract_lines(pdf_path)
    heading_re = make_heading_re(chapter_prefix)

    nodes: list[Node] = []
    node_lookup: dict[str, Node] = {}

    current = None # nó sendo acumulado no momento
    current_section = None # nó da seção aberta (para "chapter_path" e detecção
    in_anexo = None # (roman, doc_num) quando dentro de um bloco de anexo
    in_glossario = False

    def flush(n):

```

```

if n is not None:
    n.content = n.content.strip()
    # só agora, com o texto completo acumulado, decide se o nó é um
    # cabeçalho de seção (title) ou um item de conteúdo (content).
    # Anexos e Glossário já vêm com title fixado – não reavaliar.
    if n.depth is not None and not n.chapter_number.startswith(("ANEXO",
        if is_section_header(n.content):
            n.title = n.content
            n.content = ""
    nodes.append(n)
    node_lookup[n.node_id] = n

doc_prefix_id = document_id.replace("-", "")

for line in lines:
    # --- ANEXO ---
    anexo_m = ROMAN_ANEXO_RE.match(line)
    if anexo_m:
        flush(current)
        current = None
        roman = anexo_m.group(1)
        node_id = f"{doc_prefix_id}_ANEXO_{roman}"
        n = Node(
            node_id=node_id,
            document_id=document_id,
            document_type="NR",
            chapter_number=f"ANEXO {roman}",
            depth=0,
            chapter_path=f"ANEXO {roman}",
            title=None,
            content=line,
            has_table=False,
        )
        current = n
        in_anexo = roman
        in_glossario = False
        continue

    # --- GLOSSÁRIO ---
    if GLOSSARIO_RE.match(line):
        flush(current)
        node_id = f"{doc_prefix_id}_GLOSSARIO"
        current = Node(
            node_id=node_id,
            document_id=document_id,
            document_type="NR",
            chapter_number="GLOSSARIO",

```

```

        depth=0,
        chapter_path="GLOSSARIO",
        title="Glossário",
        content="",
        has_table=False,
    )
    in_glossario = True
    in_anexo = None
    continue

# --- item numerado (só fora de Anexo/Glossário: lá a numeração não segue
if not in_anexo and not in_glossario:
    m = heading_re.match(line)
    # falso positivo comum: uma citação numérica no meio de uma frase
    # ("...previsto nos subitens 13.4.1.8, 13.5.1.7 e 13.7.1.2") cai
    # sozinha no início de uma linha quebrada pelo PDF e parece um
    # novo item. Regra: um chapter_number só existe uma vez no
    # documento – se já foi visto, isso é continuação, não heading novo.
    candidate_node_id = f"{doc_prefix_id}_{m.group(1)}" if m else None
    if m and (candidate_node_id in node_lookup or (current is not None and
        current = current # mantém nó atual aberto
        m = None # trata a linha inteira como continuação abaixo
    if m:
        flush(current)
        chapter_number = m.group(1)
        rest = m.group(2)
        depth = chapter_number.count(".")
        path_parts = chapter_number.split(".")
        chapter_path = "> ".join(
            ".".join(path_parts[: i + 1]) for i in range(len(path_parts))
        )
        node_id = f"{doc_prefix_id}_{chapter_number}"

        # a decisão título-vs-conteúdo só é tomada no flush(), depois de
        # acumular TODAS as linhas do item – decidir com base só na
        # primeira linha física quebra frases que continuam na linha
        # seguinte (ex.: item 13.5.1.2.1 virava "título" truncado)
        current = Node(
            node_id=node_id, document_id=document_id, document_type="NR",
            chapter_number=chapter_number, depth=depth, chapter_path=chap
            title=None, content=rest, has_table=False,
        )
    continue

# --- linha de continuação (letras a/b/c ou wrap de texto) ---
if current is not None:
    if "Tabela" in line:

```

```

        current.has_table = True
        sep = " " if current.content and not current.content.endswith("\n",)
        current.content = (current.content + sep + line).strip()

flush(current)

# segunda passada: extrair referências agora que node_lookup está completo
for n in nodes:
    n.references = extract_references(n.content, document_id, node_lookup)

return nodes

if __name__ == "__main__":
    pdf_path = sys.argv[1] if len(sys.argv) > 1 else "/mnt/user-data/uploads/NR-1"
    document_id = sys.argv[2] if len(sys.argv) > 2 else "NR-13"
    chapter_prefix = sys.argv[3] if len(sys.argv) > 3 else "13"

    nodes = parse_nr(pdf_path, document_id, chapter_prefix)

    out = {
        "_descricao": f"Árvore extraída automaticamente de {document_id} via pars
        "_total_nodes": len(nodes),
        "nodes": [asdict(n) for n in nodes],
    }

    out_path = f"/home/claude/{document_id.replace('-', '_').lower()}_tree.json"
    with open(out_path, "w", encoding="utf-8") as f:
        json.dump(out, f, ensure_ascii=False, indent=2)

    print(f"OK: {len(nodes)} nós extraídos -> {out_path}")

```